

Lab Session 01

Explore the usage of any documentation tool in Software Development Life Cycle (SDLC)

Software documentation

All large software development projects, irrespective of application, generate a large amount of associated documentation. A high proportion of software process costs is incurred in producing this documentation. Furthermore, documentation errors and omissions can lead to errors by endusers and consequent system failures with their associated costs and disruption. Therefore, managers and software engineers should pay as much attention to documentation and its associated costs as to the development of the software itself. The documents associated with a software project and the system being developed, have a number of associated requirements:

1. They should act as a communication medium between members of the development team.
2. They should be a system information repository to be used by maintenance engineers.
3. They should provide information for management to help them plan, budget and schedule the software development process.
4. Some of the documents should tell users how to use and administer the system.

Types of Software Documentation

Generally, the software project documentation produced falls into two classes:

1. **Process documentation:** These documents record the process of development and maintenance. *Plans, schedules, process quality documents and organizational and project standards are process documentation.* The major characteristic of process documentation is that most of it becomes outdated. Plans may be drawn up on a weekly, fortnightly or monthly basis. Progress will normally be reported weekly. Although of interest to software historians, much of this process information is of little real use after it has gone out of date and there is not normally a need to preserve it after the system has been delivered.
2. **Product documentation:** This documentation describes the product that is being developed. *System documentation describes the product from the point of view of the engineers developing and maintaining the system; user documentation provides a product description that is oriented towards system users.* Unlike most process documentation, it has a relatively long life. It must evolve in step with the product, which it describes. Product documentation includes user documentation, which tells users how to use the software product and system documentation, which is principally intended for maintenance engineers. See Figure 2.1 for different types of User Documents.

System documentation consists of the following:

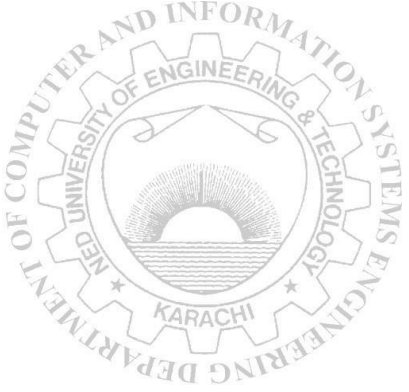
- Software Requirement Specification (SRS)
- Design and Architecture
- Source Code Documents
- Testing Documents
- Formal Technical Reviews

User documentation includes the following:

- End-users Documentation
- System-admin Documentation

Document structure

The document structure is the way in which the material in the document is organized into chapters and, within these chapters, into sections and subsections. Document structure has a major impact on readability and usability. As with software systems, you should design document structures so that the different parts are as independent as possible. The IEEE standard for user documentation proposes that the structure of a document should include the components shown in Figure 2.2.

Component	Description
Identification data	Data such as a title and identifier that uniquely identifies the document.
Table of contents	Chapter/section names and page numbers.
List of illustrations	Figure numbers and titles
Introduction	Defines the purpose of the document and a brief summary of the contents 
Information for use of the documentation	Suggestions for different readers on how to use the documentation effectively.
Concept of operations	An explanation of the conceptual background to the use of the software.

Procedures	Directions on how to use the software to complete the tasks that it is designed to support.
Information on software commands	A description of each of the commands supported by the software.
Error messages and problem resolution	A description of the errors that can be reported and how to recover from these errors.
Glossary	Definitions of specialized terms used.
Related information sources	References or links to other documents that provide additional information
Navigational features	Features that allow readers to find their current location and move around the document.
Index	A list of key terms and the pages where these terms are referenced.
Search capability	In electronic documentation, a way of finding specific terms in the document.

Fig 1.1: Suggested components in a software user document

Document Preparation

Document preparation is the process of creating a document and formatting it for publication. Figure 2.3 shows the document preparation process as being split into 3 stages namely document creation, polishing and production. The three phases of preparation and associated support facilities are explained in the figure below:

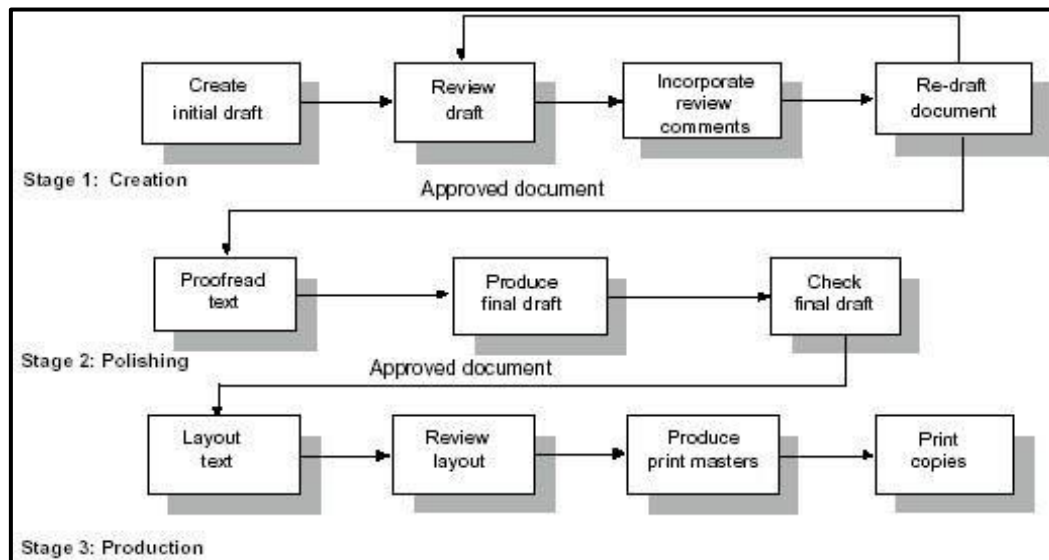


Fig 1.2: Stages of document preparation

LaTeX

LaTeX is a high-quality document preparation system; it includes features designed for the production of technical and scientific documentation. It allows users to very quickly tackle the more complicated parts of typesetting, such as inputting mathematics, creating tables of contents, referencing and creating bibliographies, and having a consistent layout across all sections. It is based on the WYSIWYM (what you see is what you mean) idea, meaning you only have focus on the contents of your document and the computer will take care of the formatting.

LaTeX is available as free software. To set up a basic TeX/LaTeX system, download and run the Basic MiKTeX Installer. TeXworks is used as editor for writing commands. We start off with a basic .tex file which contains LaTeX code. LaTeX uses control statements, which define how your content should be formatted. LaTeX compiler takes .tex file as input and convert it into a .pdf file.

First Piece of LaTeX

```

\documentclass{article}

\begin{document}
First document. This is a simple example, with no extra
parameters or packages included.
\end{document}

```

The first line of code declares the type of document, known as the *class*. The class controls the overall appearance of the document. In this case, the class is article, the simplest and most common LATEX

class. Other types of documents you may be working on may require different classes such as **book** or **report**.

Then the contents of our document are written, enclosed inside the `\begin{document}` and `\end{document}` tags. This is known as the *body* of the document. You can start writing here and make changes to the text if you wish. To see the result of these changes in the PDF you have to compile the document.

The Preamble of a document

In the previous example the text was entered after the `\begin{document}` command. Everything in your `.tex` file *before* this point is called the **preamble**. In the preamble you define the type of document you are writing, the language you are writing in, the *packages* you would like to use (more on this later) and several other elements. For instance, a normal document preamble would look like this:

```
\documentclass[12pt, letterpaper]{article}
```

Below a detailed description of `\documentclass`:

```
\documentclass[12pt, letterpaper]{article}
```

As said before, this defines the type of document. Some additional parameters included in the square brackets can be passed to the command. These parameters must be comma-separated. In the example, the extra parameters set the font size (**12pt**) and the paper size (**letterpaper**). Of course other font sizes (**9pt**, **11pt**, **12pt**) can be used, but if none is specified, the default size is **10pt**. As for the paper size other possible values are **a4paper** and **legalpaper**.

Adding a title, author and date

To add a title, author and date to our document, you must add three lines to the **preamble** (NOT the main body of the document). These lines are:

```
\title{First document}
```

```
\author{Hubert Farnsworth}
```

```
\thanks{funded by the Overleaf team}
```

This can be added after the name of the author, inside the braces of the **title** command. It will add a superscript and a footnote with the text inside the braces.

```
\date{September 2019}
```

Enter the date manually or use the command `\today` so the date will be updated automatically.

```
\documentclass[12pt, letterpaper, twoside]{article}

\title{First document}
\author{Hubert Farnsworth \thanks{funded by the Overleaf team}}
\date{September 2019}
```

Now a title, author and date are provided to the document, we can print this information on the document with the **\maketitle** command. This should be included in the **body** of the document at the place you want the title to be printed.

```
\begin{document}

\maketitle
We have now added a title, author and date to our first \LaTeX{}
document! \end{document}
```

Adding Comments

To make a comment in LATEX, simply write a % symbol at the beginning of the line as shown below:

```
\begin{document}

\maketitle
\pagenumbering {gobble}
We have now added a title, author and date to our first \LaTeX{}
document!
% This line here is a comment. It will not be printed in the
document.
\end{document}
```

Basic Formatting

• Abstracts

In scientific documents it's a common practice to include a brief overview of the main subject of the paper. In LATEX there's the **abstract** environment for this. The **abstract** environment will put the text in a special format at the top of your document.

```
\begin{document}
\newpage
\begin{abstract}
This is a simple paragraph at the beginning of the
document. A brief introduction about the main subject.
\end{abstract}
\end{document}
```

• Paragraphs and Newlines

When writing the contents of your document, if you need to start a new paragraph you must hit the "Enter" key twice (to insert a double blank line). Notice that LATEX automatically indents paragraphs. To start a new line without actually starting a new paragraph insert a break line point, this can be done by \\ (a double backslash as in the example) or the \newline command.

```
\begin{document}
```

```

\begin{abstract}
This is a simple paragraph at the beginning of the document. A
brief introduction about the main subject.
\end{abstract}
Now that we have written our abstract, we can begin writing our
first paragraph.

This line will start a second Paragraph.
\end{document}

```

• Chapters and Sections

Commands to organize a document vary depending on the document type, the simplest form of organization is the sectioning, available in all formats.

```

\chapter{First Chapter}

\section{Introduction}

This is the first section.

Lorem ipsum dolor sit amet, consectetur adipiscing
elit. Etiam lobortis facilisis sem. Nullam nec mi et
neque pharetra sollicitudin. Praesent imperdiet mi nec
ante. Donec ullamcorper, felis non sodales...

\section{Second Section}

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam
lobortis facilisissem. Nullam nec mi et neque pharetra
sollicitudin. Praesent imperdiet mi nec ante...

\subsection{First Subsection}
Praesent imperdiet mi nec ante. Donec ullamcorper, felis non
sodales...

\section*{Unnumbered Section}
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Etiam lobortis facilisissem

```

The command `\section{}` marks the beginning of a new section, inside the braces is set the title. Section numbering is automatic and can be disabled by including a `*` in the section command as `\section*{}`. We can also have `\subsection{}`s, and indeed `\subsubsection{}`s. The basic levels of depth are listed below:

-1 `\part{part}`

- 0 `\chapter{chapter}`
- 1 `\section{section}`
- 2 `\subsection{subsection}`
- 3 `\subsubsection{subsubsection}`
- 4 `\paragraph{paragraph}`
- 5 `\subparagraph{subparagraph}`

Note that `\part` and `\chapter` are only available in report and book document classes.

• Bold, Italics and Underlining

Following are some simple text formatting commands.

- **Bold:** Bold text in LaTeX is written with the `\textbf{...}` command.
- *Italics:* Italicised text in LaTeX is written with the `\textit{...}` command.
- Underline: Underlined text in LaTeX is written with the `\underline{...}` command.

Some of the `\textbf{greatest}` discoveries in `\underline{science}` were made by `\textbf{\textit{accident}}`.

• Unordered lists

Unordered lists are produced by the `itemize` environment. Each entry must be preceded by the control sequence `\item` as shown below. By default the individual entries are indicated with a black dot, so-called bullet. The text in the entries may be of any length.

```
\begin{itemize}
  \item The individual entries are indicated with a black dot, a
  so-called bullet.
  \item The text in the entries may be of any length.
\end{itemize}
```

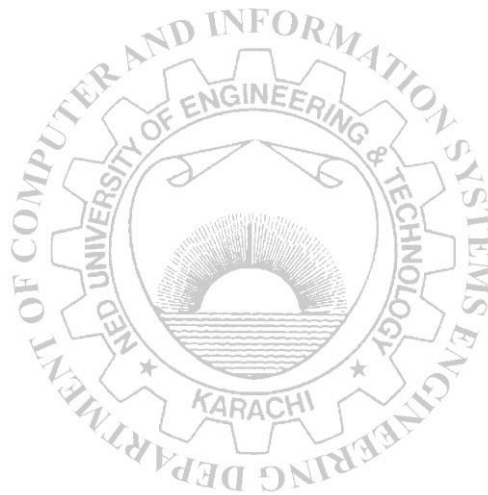
• Ordered lists

Ordered lists have the same syntax inside a different environment. We make ordered lists using the `enumerate` environment:

```
\begin{enumerate}
  \item This is the first entry in our list
  \item The list numbers increase with each entry we add
\end{enumerate}
```


EXERCISES

1. Construct a Software Requirement Specifications (SRS) document for a Software System of your choice using LaTeX. Attach the printout of the .tex file and .pdf file.
2. Develop a brief document describing the basic functionality of an IoT based Health Monitoring System. Also include any block diagram/ image for explaining the overall functionality of the system. Use LaTeX. Attach the printout of the .tex and .pdf files. (Feel free to you use the other available formatting options in LaTeX).
3. Suppose that you are developing a calculator application and gathering requirements for it. Your calculator is able to solve some basic linear equations as well. Prepare an initial draft of requirement specification document, highlighting various examples of equations your system is able to solve. Use LaTeX. Attach the printout of the .tex and .pdf files.



Software Requirement Specifications

For RideNED

Version # 01

Prepared by Group ID: A4

Raazia Imran (SE-24009)
Muzammil Ali (SE-24034)
Rameez Kamran (SE-24047)
Abdullah Zia (SE-24048)

14 April 2025

Revision History

Req ID	Date	Reason for Change	Version

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Document Convention	4
1.3	Intended Audience and Reading Suggestions	4
1.4	Product Scope	4
2	Overall Description	4
2.1	Product Perspective	4
2.2	Product Functions	4
2.3	User Classes and Characteristics	4
2.3.1	NED University Students	5
2.3.2	NED Admin	5
2.4	Operating Environment	5
2.5	User Documentation	5
2.6	Assumptions and Dependencies	5
3	System Features	6
3.1	University Point Tracking System	6
3.1.1	Description and Priority	6
3.1.2	Response Sequence	6
3.1.3	Functional Requirements	6
3.1.4	Non-Functional Requirements	7
3.2	Report Lost/Found Items	7
3.2.1	Description and Priority	7
3.2.2	Response Sequence	7
3.2.3	Functional Requirements	7
3.2.4	Non-Functional Requirements	7
3.3	Digital Point Fee Payment	7
3.3.1	Description and Priority	8
3.3.2	Response Sequence	8
3.3.3	Functional Requirements	8
3.3.4	Non-Functional Requirements	8
3.4	Digital Point Pass	8
3.4.1	Description and Priority	8
3.4.2	Functional Requirements	9
3.4.3	Non-Functional Requirements	9
3.5	Point Pre-Booking System	9
3.5.1	Description and Priority	9
3.5.2	Functional Requirements	9
3.5.3	Non-Functional Requirements	9
3.6	Driver Complaint System	9
3.6.1	Description and Priority	10
3.6.2	Functional Requirements	10
3.6.3	Non-Functional Requirements	10
3.7	Admin Dashboard	10
3.7.1	Description and Priority	10

3.7.2	Functional Requirements	10
3.7.3	Non-Functional Requirements	10
4	Other Non-Functional Requirements	11
4.1	Performance Requirements	11
4.2	Database Requirements	11
4.3	Safety Requirements	11

1 Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the **RideNED** application. This document lists the functional and non-functional requirements and creates a clear understanding between the stakeholders and the developers regarding the system features.

1.2 Document Convention

This document follows the IEEE standard for Software Requirement Specifications. Requirements are prioritized as High, Medium, or Low.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- **Developers:** To understand the features to be implemented.
- **Testers:** To validate the system against requirements.
- **Project Managers:** To track progress and scope.
- **Stakeholders:** To verify that their needs are met.

1.4 Product Scope

RideNED is a mobile application designed to facilitate the transportation system at NED University. It digitizes the process of tracking points, paying fees, and managing passes.

2 Overall Description

2.1 Product Perspective

The product is not independent and not a standalone system as it is a part of the larger NED University's transport management system. It relies on:

- The university's existing student authentication system.
- Google Maps services for location tracking.
- Hardware scanners for marking point attendance digitally.

2.2 Product Functions

Note: Detailed functions are listed in Section 3 (System Features).

2.3 User Classes and Characteristics

The following are the main users of this application:

2.3.1 NED University Students

The NED student is an entity that uses the mobile application to access transportation-related services.

- **Capabilities:** Track live point locations, report complaints, report lost/found items, book seats, access digital bus passes, and pay point fees.
- **Prerequisites:** Must have university account credentials. Expected to be familiar with basic mobile navigation, online payments, and QR codes.
- **Permissions:** Must allow microphone access (voice assistant) and location services.

2.3.2 NED Admin

The admin has access to the complete features of the application.

- **Responsibilities:** Manage overall operations, update routes, monitor locations, manage lost/found reports, and upload trip data.
- **Skills:** Efficient in managing digital dashboards, data organization, and responding to user requests accurately.

2.4 Operating Environment

The project is semi-automated. It automates features like live tracking and digital passes but requires administrative tasks (managing routes, handling complaints) ensuring human oversight for critical decisions.

2.5 User Documentation

Student Interface:

- Install app and grant permissions (location, storage).
- Register using university credentials and verify via OTP (Gmail).
- Access dashboard for tracking, fees, and forms.

Admin Interface:

- Register as admin using valid university credentials and verify via OTP.
- Access admin panel to manage routes, complaints, statistics, and transactions.

2.6 Assumptions and Dependencies

- Students possess valid university-issued credentials.
- Points are operational based on provided schedules.
- Points are equipped with “Eyoyo USB QR Code Scanners.”
- Google Maps services are available for tracking.
- Students use Android/iOS smartphones with GPS enabled.

3 System Features

3.1 University Point Tracking System

Author: Rameez Kamran [SE-24047]

3.1.1 Description and Priority

This feature helps students view the real-time location of university points on a map using GPS. It addresses the issue of uncertainty regarding arrival times.

Priority: High

3.1.2 Response Sequence

- User logs in and selects “Routes” from the dashboard.
- System displays list of points (R1, R2, etc.).
- User selects a route; system fetches GPS location from the tracker.
- System displays live location on the map, refreshing continuously.

3.1.3 Functional Requirements

FT REQ-1	Users can sort points by route number, bookmarked routes, or ‘optimized suggestion’ based on device GPS.
FT REQ-2	User can view the list of stops for a point with exact locations on the map.
FT REQ-3	User can bookmark up to three points for dashboard access. Oldest bookmark is replaced if limit exceeded.
FT REQ-4	‘Notify Me’ option sends a message when a point is 5 minutes away from a selected stop.
FT REQ-5	Home screen widget displays live location.
FT REQ-6	Map view defaults to roadmap but supports satellite view.
FT REQ-7	User can share point location/ETA via WhatsApp (includes map link).
FT REQ-8	ETA for stops updates automatically based on traffic data.
FT REQ-9	User can view driver details (name, picture, route).
FT REQ-10	Voice input support for visually impaired users (e.g., “Where is route 3?”).
FT REQ-11	In-built voice assistant requires microphone permission.

3.1.4 Non-Functional Requirements

- FT REQ-4.1** Popups disappear automatically in 10 seconds.
- FT REQ-5.1** Only one tracking widget allowed at a time.
- FT REQ-5.2** Widget does not auto-update to save battery (manual refresh required).
- FT REQ-7.1** Driver image must be passport-sized, JPEG/PNG, blue background, < 1MB.
- FT REQ-9.1** Driver info fetch time must not exceed 10 seconds.

3.2 Report Lost/Found Items

Author: Rameez Kamran [SE-24047]

3.2.1 Description and Priority

A centralized platform for reporting and recovering lost items on university points.

Priority: Medium

3.2.2 Response Sequence

- User selects “Report Lost/Found Item”.
- User fills form (description, route, photo).
- Item is posted to Lost and Found Board.
- Matched users connect via in-app chat. Item removed when marked “Returned”.

3.2.3 Functional Requirements

- FT REQ-12** Form requires item name, description, date, route, and optional image.
- FT REQ-13** User marks item as “claimed” by describing a hidden detail.
- FT REQ-14** Found items are public; Lost items are private until a match is found.

3.2.4 Non-Functional Requirements

- FT REQ-14.1** Contact numbers hidden; chat history refreshes 48 hours after claim.
- FT REQ-12.1** Description limit 150 chars; Image limit 1MB.
- FT REQ-12.5** Anti-spam: Max 1 report per day.

3.3 Digital Point Fee Payment

Author: Raazia Imran [SE-24009]

3.3.1 Description and Priority

Allows students to pay point fees via E-wallet or Bank Transfer and upload proof for admin verification.

Priority: High

3.3.2 Response Sequence

- Student selects payment method.
- System checks eligibility.
- Student pays and uploads screenshot/receipt.
- Admin verifies; if approved, student proceeds to pass generation.

3.3.3 Functional Requirements

- FT REQ-1** Supports E-wallet (EasyPaisa/JazzCash) and Bank Transfer (NBP/HMB).
- FT REQ-2** Wallet: Requires Transaction ID and screenshot.
- FT REQ-3** Bank: Displays account details; requires Ref Number and receipt.
- FT REQ-4** Admin interface for approving/rejecting payments.
- FT REQ-5** “Proceed to Pass Generation” enabled upon approval.
- FT REQ-6** Failed payments logged with error codes.

3.3.4 Non-Functional Requirements

- FT REQ-1.1** Only registered on-campus students are eligible.
- FT REQ-2.1** Proofs stored in secure cloud bucket (AES-256 encrypted).
- FT REQ-4.1** Verification completion target: 2 hours (working hours).

3.4 Digital Point Pass

Author: Raazia Imran [SE-24009]

3.4.1 Description and Priority

Generates a QR-based monthly pass tied to Student ID, replacing manual checks.

Priority: High

3.4.2 Functional Requirements

- FT REQ-7** access allowed only if semester fee is “Paid”.
- FT REQ-8** Mandatory fields: ID, Name, Phone (OTP), Dept, Batch, Photo.
- FT REQ-10** Pass contains details and encrypted QR code.
- FT REQ-12** OS-level restrictions on screenshots/recording.
- FT REQ-14** Countdown timer for expiry; renewal allowed with < 7 days remaining.
- FT REQ-15** History of past passes and payments.

3.4.3 Non-Functional Requirements

- FT REQ-1.2** Daily jobs check validity and send expiry reminders (3 days prior).
- FT REQ-10.1** Unique QR hash regenerated every 24 hours.
- FT REQ-11.1** Pass accessible offline.

3.5 Point Pre-Booking System

Author: Muzammil Ali [SE-24034]

3.5.1 Description and Priority

Allows booking of Department-Specific or General trips with seat management.

Priority: Medium

3.5.2 Functional Requirements

- FT REQ-1** Trips categorized as “Department-Specific” or “General”.
- FT REQ-3** System filters trips based on student department.
- FT REQ-6** Support for trips requiring special approval (e.g., HOD).
- FT REQ-9** Prevention of overlapping bookings.
- FT REQ-10** Live seat counters.

3.5.3 Non-Functional Requirements

- FT REQ-1.1** 100% accurate filtering of eligibility.
- FT REQ-4.1** Consistent color-coded badges for trip categories.

3.6 Driver Complaint System

Author: Muzammil Ali [SE-24034]

3.6.1 Description and Priority

Streamlines reporting of unsafe driving or behavior with anonymous options.

Priority: Medium

3.6.2 Functional Requirements

- FT REQ-14** Auto-fills trip details (Vehicle, Driver).
- FT REQ-16** Anonymous submission option (identity hidden from driver).
- FT REQ-17** Progress timeline (Under Review, Resolved).
- FT REQ-20** Auto-save drafts every 15 seconds.
- FT REQ-21** Tone analyzer warns against abusive language.

3.6.3 Non-Functional Requirements

- FT REQ-18.1** Media upload: JPG, PNG, MP3 (< 10MB).

3.7 Admin Dashboard

Author: Abdullah Zia [SE-24048]

3.7.1 Description and Priority

Control panel for managing points, passes, announcements, and statistics.

Priority: High

3.7.2 Functional Requirements

- FT REQ-1** View and sort user complaints.
- FT REQ-2** Generate user point usage reports (PDF).
- FT REQ-6** Add/Remove routes and stops.
- FT REQ-7** Cancel suspicious digital passes.
- FT REQ-10** Emergency alert for route deviation.
- FT REQ-17** Notification templates (Pass Expiring, Maintenance).
- FT REQ-21** Password re-entry required for critical deletions.

3.7.3 Non-Functional Requirements

- FT REQ-2.1** PDF generation < 5 seconds.
- FT REQ-13.1** Admin changes reflect in < 5 seconds.

4 Other Non-Functional Requirements

4.1 Performance Requirements

The app must be optimally responsive across devices (smartphones, tablets). This includes fast loading times, smooth animations, and seamless transitions.

4.2 Database Requirements

A NoSQL database (e.g., MongoDB) will be used to handle real-time data, including:

- Booking data (Student IDs, Trip IDs).
- Trip data (Routes, Seat availability).
- Real-time bus location (GPS coordinates) and QR code data.

4.3 Safety Requirements

- Scanner cables must be insulated to prevent electrical issues.
- Secure connection between scanner and device to prevent data loss.
- Scanners must contain overload protection and auto-shutdown features for overheating.

IoT-Based Health Monitoring System

Functional Description & Architecture

Proposed by:

Raazia Imran

2nd-Year Software Engineering Student, NED University

January 14, 2026

1 Introduction

The **IoT-Based Health Monitoring System** is a smart healthcare solution designed to track patient vital signs in real-time and transmit data to a remote server. This system aims to bridge the gap between patients and medical staff by reducing the need for physical hospital visits for routine checkups.

Using a network of sensors and internet connectivity, the system ensures that critical health parameters such as heart rate, body temperature, and oxygen saturation (SpO_2) are monitored continuously.

2 System Architecture

The following block diagram illustrates the data flow from the patient to the medical interface.

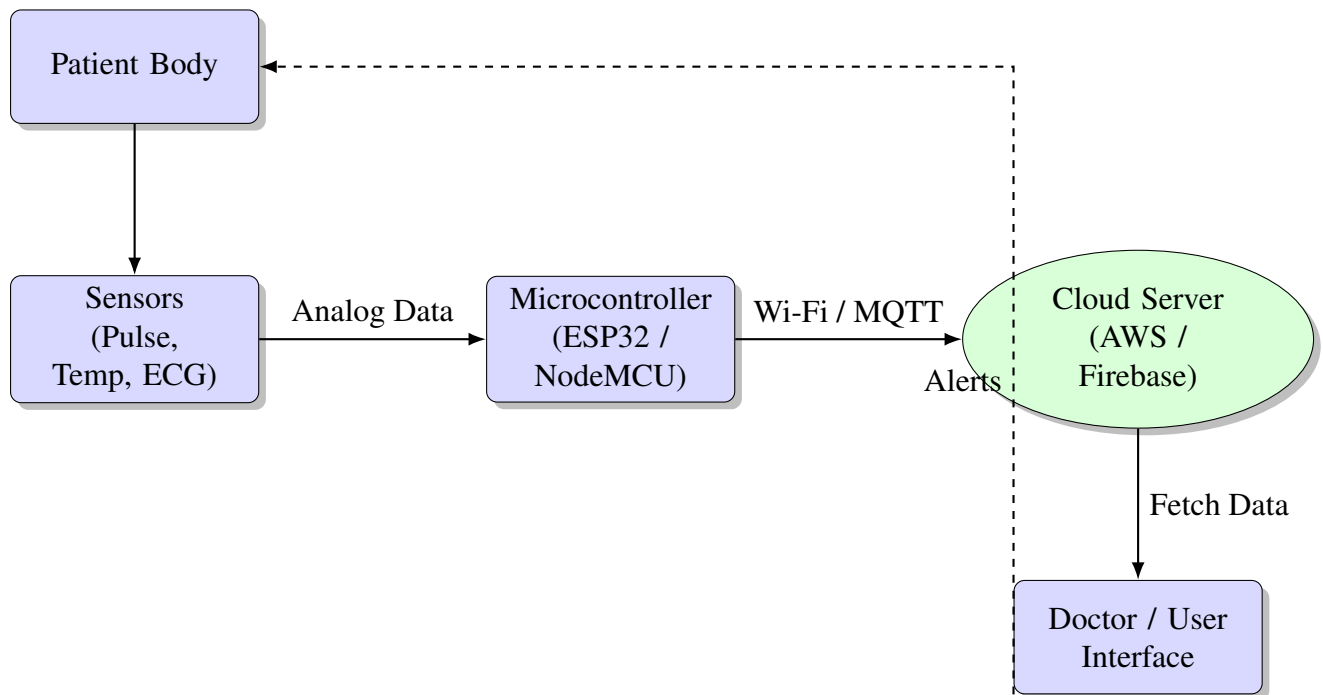


Figure 1: Block Diagram of IoT Health Monitoring System

3 Functional Modules

The system is divided into three primary layers: the Sensing Layer, the Processing Layer, and the Application Layer.

3.1 1. Sensing Layer

This layer consists of non-invasive hardware sensors attached to the patient.

- **Pulse Sensor:** Measures the heart rate in Beats Per Minute (BPM).
- **LM35 / DHT11:** Measures body temperature.
- **MAX30100:** Measures blood oxygen levels (SpO_2).

3.2 2. Processing and Transmission Layer

The core processing unit (e.g., Arduino or ESP32) collects raw data from the sensors.

Data Processing Logic

The microcontroller reads analog signals, converts them to digital values, and checks against predefined thresholds (e.g., Body Temp $> 38^{\circ}\text{C}$). If a threshold is breached, an immediate local alert is triggered.

3.3 3. Cloud and Application Layer

Data is sent over Wi-Fi using protocols like **MQTT** or **HTTP** to a cloud platform (e.g., ThinkSpeak or Firebase).

- **Dashboard:** Doctors can view live graphs of patient health.
- **Alert System:** If the cloud detects an anomaly, it sends an SMS or Email notification to the doctor and the patient's family.

4 Conclusion

The proposed IoT-based system provides a cost-effective and reliable method for remote patient monitoring. By automating the tracking of vital signs, it reduces the burden on hospital resources and ensures timely medical intervention in emergencies.

Software Requirement Specifications

SmartCalc: Algebraic Solver Edition

Prepared For:

Professor / Course Supervisor

Prepared By:

Raazia Imran

2nd-Year Software Engineering Student, NED University

Version 1.0

January 14, 2026

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions and Acronyms	1
2	Overall Description	1
2.1	Product Perspective	1
2.2	User Classes	1
3	System Features	1
3.1	Standard Arithmetic Engine	1
3.2	Linear Equation Solving Engine	2
3.2.1	Input Constraints	2
3.2.2	Solved Equation Examples	2
4	External Interface Requirements	3
4.1	User Interface (UI)	3
5	Non-Functional Requirements	3
5.1	Performance	3
5.2	Robustness	3

1 Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for **SmartCalc**. This application is designed to bridge the gap between standard arithmetic calculators and complex graphing tools by offering an intuitive **Linear Equation Solver**.

1.2 Scope

The software is a mobile application (Android/iOS) that allows users to:

1. Perform standard arithmetic calculations.
2. Input linear algebraic equations using a specialized keypad.
3. Receive instant solutions for the variable x .

1.3 Definitions and Acronyms

- **SRS:** Software Requirement Specification.
- **PEMDAS:** Order of operations (Parentheses, Exponents, Multiplication, Division, Addition, Subtraction).
- **Linear Equation:** An equation of the first degree (e.g., $ax + b = c$).

2 Overall Description

2.1 Product Perspective

SmartCalc operates as a standalone application. It does not require internet connectivity for its solving engine, ensuring availability in exam or field environments (subject to academic integrity policies).

2.2 User Classes

- **Students:** Primary users requiring assistance with algebra homework.
- **General Users:** Users needing quick arithmetic.

3 System Features

3.1 Standard Arithmetic Engine

Description: The system shall function as a standard calculator.

Functional Requirement 1.0: The system shall accept inputs for numbers (0-9) and operators (+, -, ×, ÷).

Functional Requirement 1.1: The system shall handle floating-point results up to 10 decimal places.

3.2 Linear Equation Solving Engine

Description: The system shall parse string inputs containing the variable 'x' and the equality operator '=' to solve for 'x'.

3.2.1 Input Constraints

- The equation must contain exactly one variable type (x).
- The equation must contain exactly one equality sign (=).
- The equation degree must not exceed 1 (Linear only, no x^2).

3.2.2 Solved Equation Examples

The system is explicitly required to solve the following mathematical patterns.

Case A: Standard Form ($ax + b = c$)

The solver must handle basic integer coefficients.

$$3x + 15 = 30 \quad (1)$$

System Output: $x = 5$

Case B: Variables on Both Sides

The solver must group like terms from both sides of the equality.

$$5x - 10 = 2x + 14 \quad (2)$$

Logic: $5x - 2x = 14 + 10 \rightarrow 3x = 24$.

System Output: $x = 8$

Case C: Parentheses & Distribution

The system must apply the distributive property before solving.

$$2(x - 3) + 4 = 10 \quad (3)$$

Logic: $2x - 6 + 4 = 10 \rightarrow 2x - 2 = 10 \rightarrow 2x = 12$.

System Output: $x = 6$

Case D: Fractional Coefficients

The system must handle rational numbers.

$$\frac{1}{2}x + \frac{1}{4} = \frac{3}{4} \quad (4)$$

System Output: $x = 1$

4 External Interface Requirements

4.1 User Interface (UI)

- **Screen 1 (Calc):** Standard numeric keypad.
- **Screen 2 (Algebra):** Extended keypad adding keys: $[x]$, $[(], [)], [=]$.
- **Result Display:** A distinct display area showing the step, e.g., " $x = 5$ ".

5 Non-Functional Requirements

5.1 Performance

- **NFR-1:** Equation solving time shall not exceed 0.5 seconds on standard devices.
- **NFR-2:** App launch time shall be under 2 seconds.

5.2 Robustness

- **NFR-3:** If a user inputs a non-linear equation (e.g., $x^2 + 2 = 0$), the system shall return a "Feature Not Supported" error rather than crashing.
- **NFR-4:** Infinite solution cases (e.g., $2x = 2x$) shall display "Infinite Solutions".